

PERFORMANCE EVALUATION AND DIMENSIONING OF NETWORKS AND SYSTEMS

METI

Lab 2 Report [EN]

Authors:

José Oliveira (103771)

Afonso Mendes (116024)

Tiago Videira (116069)

jose.novais.oliveira@tecnico.ulisboa.pt

afonsofmendes@tecnico.ulisboa.pt

tiago.g.videira@tecnico.ulisboa.pt

2026/2027 – 1st Semester

Contents

1	Introduction	2
2	A. Discrete-time Markov chains	3
2.1	Bernoulli and 2-DTMC sample paths	3
2.2	Estimating transitions from the attack trace	4
2.3	Slotted ALOHA	5
2.3.1	Theoretical throughput	5
2.3.2	Discrete-event simulation	6
2.3.3	Comparison with CSMA and CSMA/CD	7
3	Conclusion	7

1 Introduction

This report presents the solutions to the three exercises assigned to class 5 in Laboratory 2 of the Performance Evaluation and Dimensioning of Networks and Systems (DDRS) course. It follows the objective–approach–results–conclusion structure used in the Laboratory 1 report. The R scripts and generated figures are in the `scripts` directory; the supplied ALOHA theory function and attack trace are reused from `scripts_helper`.

2 A. Discrete-time Markov chains

2.1 Bernoulli and 2-DTMC sample paths

Objective: Compare Bernoulli processes with two-state discrete-time Markov chains (2-DTMCs) using two parameter pairs that have the same stationary mean but markedly different temporal behavior.

Approach: For a 2-DTMC with transition matrix

$$P = \begin{bmatrix} \alpha & 1 - \alpha \\ \beta & 1 - \beta \end{bmatrix},$$

the stationary probability of state 1 is

$$\pi_1 = \frac{1 - \alpha}{1 - \alpha + \beta}.$$

The script `E1_DTMCBernoulli.R` compares 200-step sample paths for $(\alpha, \beta) = (0.9, 0.1)$ and $(0.1, 0.9)$. Both have $\pi_1 = 0.5$, so each is paired with an independent Bernoulli process with success probability 0.5. A fixed random seed makes the output reproducible.

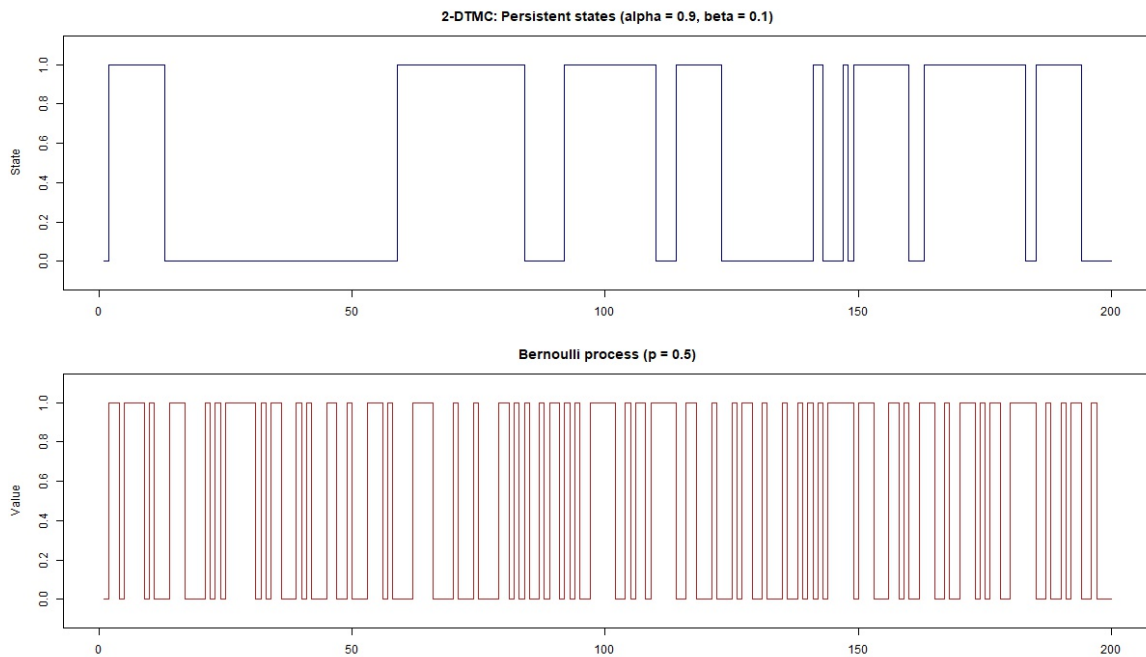


Figure 1: Persistent 2-DTMC states compared with a Bernoulli process of the same mean.

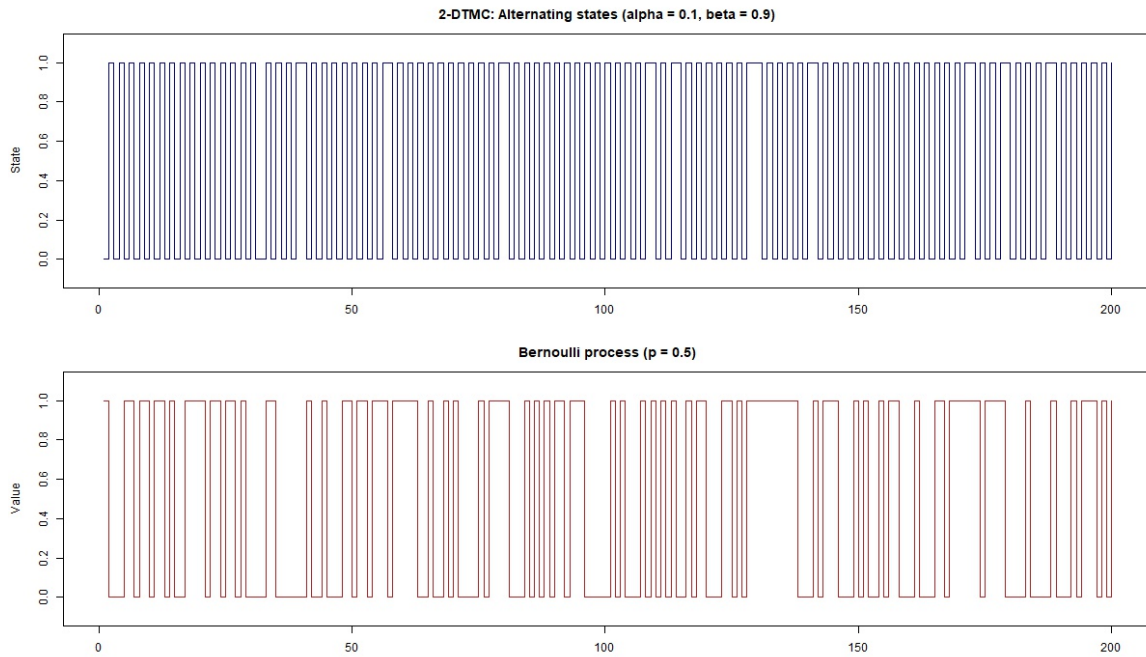


Figure 2: Alternating 2-DTMC states compared with a Bernoulli process of the same mean.

Results and conclusion: With $(0.9, 0.1)$, a state is likely to persist, so the 2-DTMC exhibits long runs of zeros or ones. With $(0.1, 0.9)$, transitions between the states are likely, producing an alternating pattern. In both cases the long-run fraction of ones is 0.5, matching the Bernoulli mean, but the Bernoulli samples have no temporal dependence. A 2-DTMC can therefore model persistence, clustering, and alternation in a sequence; an independent Bernoulli process cannot represent these dependencies.

2.2 Estimating transitions from the attack trace

Objective: Estimate the transition probabilities of a 2-DTMC from the sequence of daily server attack indicators in `2dtmcddata.txt`.

Approach: The script `E2_AttackDTMC.R` imports the supplied file with the tab separator specified in the exercise. It counts each consecutive transition $i \rightarrow j$ and divides by the number of transitions originating in state i . The trace contains 2,000 observations and thus 1,999 transitions.

Table 1: Estimated transition counts and probabilities for the attack trace.

Current state	Transition counts		Estimated probabilities	
	To 0	To 1	To 0	To 1
0 (not attacked)	623	577	0.5192	0.4808
1 (attacked)	577	222	0.7222	0.2778

Results and conclusion: The estimated transition matrix, with rows denoting the current state, is

$$\hat{P} = \begin{bmatrix} 0.5192 & 0.4808 \\ 0.7222 & 0.2778 \end{bmatrix}.$$

The fitted chain estimates an attack probability of approximately 0.3997 in steady state. An attack is followed by another attack with estimated probability 0.2778; otherwise, the process returns to the non-attacked state with probability 0.7222. These are empirical estimates from this finite trace, rather than known properties of all server attacks.

2.3 Slotted ALOHA

Objective: Study the theoretical and simulated throughput of slotted ALOHA with a finite number of users, explain the effect of the parameters N , σ , and p , and compare ALOHA with carrier-sensing protocols.

2.3.1 Theoretical throughput

Approach: A system state is the number of backlogged users. A thinking user transmits with probability σ in a slot, while a backlogged user transmits with probability p . Throughput is the long-run number of successful transmissions per slot. The script `E3_SlottedALOHA.R` calls the supplied `aloha_theo()` function to evaluate the stationary throughput over the requested parameter ranges.

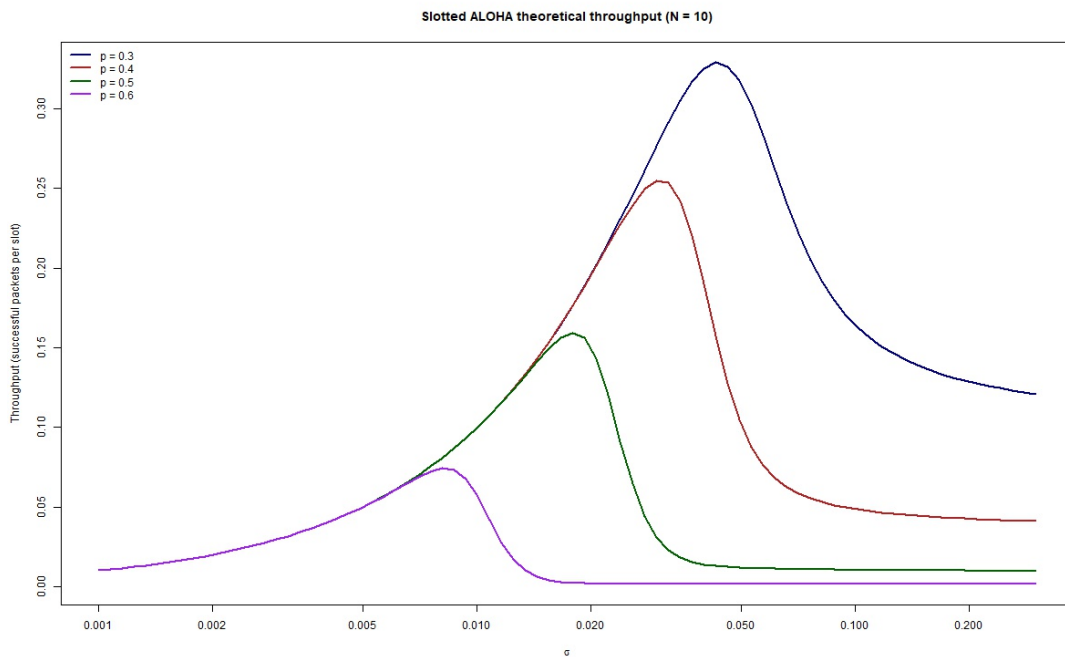


Figure 3: Theoretical slotted-ALOHA throughput for $N = 10$ and the four requested values of p .

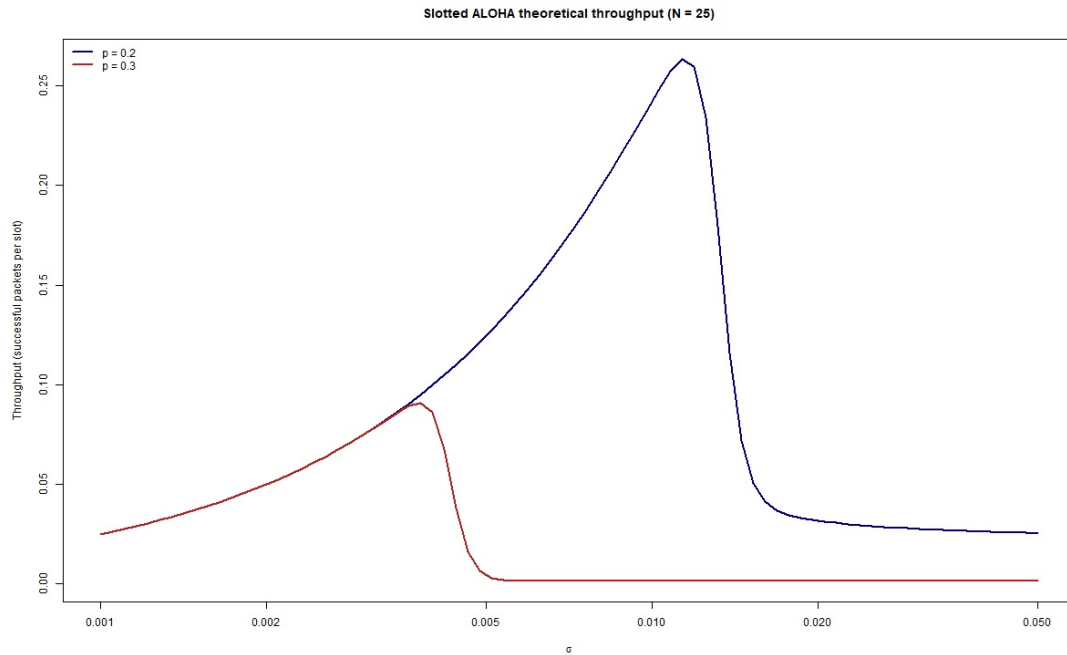


Figure 4: Theoretical slotted-ALOHA throughput for $N = 25$ and the two requested values of p .

Results and conclusion: Increasing σ causes more thinking users to attempt transmission. Initially this tends to increase throughput, but excessive attempts create collisions and move users into the backlog, so the throughput eventually peaks and can decrease. Increasing p helps clear a backlog when few backlogged users are present, but when many are backlogged it also raises the chance of collisions. Increasing N creates more potential traffic and contention; it does not guarantee higher throughput, since a larger population can spend more time in collision-heavy backlog states. The best operating point therefore depends jointly on all three parameters.

2.3.2 Discrete-event simulation

Approach: The same script runs one slot at a time. Each user attempts with the probability associated with its state. If exactly one user transmits, the slot is successful and a successful backlogged user returns to THINKING. If two or more transmit, there is a collision and each transmitting THINKING user becomes BACKLOGGED; existing backlogged users remain so. After a warm-up of 100,000 slots, throughput is estimated over 1,000,000 slots. The theoretical reference is computed by `aloha.theo()`.

Table 2: Theoretical and simulated throughput for three operating points.

N	σ	p	Theoretical	Simulation
10	0.001	0.3	0.01000	0.01004
10	0.100	0.3	0.16415	0.16309
10	0.100	0.6	0.00160	0.00157

Results and conclusion: The simulated estimates are close to their theoretical values for all three configurations. The results illustrate the sensitivity to the offered load and retransmission probability: the middle setting has the highest throughput, while raising p to 0.6 at the same σ leaves more users contending in the backlog and substantially reduces successful transmissions.

2.3.3 Comparison with CSMA and CSMA/CD

ALOHA allows a user to transmit without first sensing whether the channel is occupied, so simultaneous attempts can collide. CSMA (Carrier Sense Multiple Access) reduces collisions by sensing the medium before transmitting. CSMA/CA, used by IEEE 802.11 wireless LANs, attempts to avoid collisions because a wireless station generally cannot reliably detect a collision while it is transmitting. CSMA/CD (Carrier Sense Multiple Access with Collision Detection) detects collisions during transmission and was used by shared, half-duplex Ethernet; modern switched full-duplex Ethernet does not use CSMA/CD. ALOHA-family random-access methods are also used in RFID tag anti-collision protocols and satellite/random-access links. Carrier sensing can improve channel efficiency when stations can sense one another, while ALOHA remains useful where sensing is unavailable or impractical.

References: GS1, *EPC Radio-Frequency Identity Protocols Generation-2 UHF RFID*, for framed slotted-ALOHA tag inventory; IEEE 802.11 for wireless LAN medium access; and IEEE 802.3 for Ethernet, including the historical shared-medium collision-detection mechanism.

3 Conclusion

The three class 5 exercises demonstrate how DTMCs extend independent Bernoulli models by representing temporal dependence, how transition probabilities can be estimated directly from observed state changes, and how the stationary behavior of a contention protocol can be studied analytically and by simulation. The estimated and simulated results are consistent with the corresponding models, while also showing that finite data and operating parameters matter when interpreting performance.